

0.5.2. La même ligne, avec une boucle for — doublé, deux positions

La boucle deux fois : la ligne 0 puis la ligne 5.

1. Ce qu'il faut comprendre

C'est le 0.5 et le 0.5.1 réunis : la boucle du 0.5 (ligne 0), puis celle du 0.5.1 (ligne 5).

Deux boucles l'une après l'autre : la première finit ses vingt tours, puis la seconde commence. Chaque `for` déclare son propre `i` : il n'existe que dans sa boucle.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1394 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // 1. La ligne 0 (le 0.5)
    for (uint8_t i = 0; i < 20; i++) {    // 20 tours : les colonnes 0 à 19
        poser(i, 0, ALPHABET[i]);        // la lettre n° i, colonne i, ligne 0
    }
    // 2. La ligne 5 (le 0.5.1)
    for (uint8_t i = 0; i < 20; i++) {    // 20 tours : les colonnes 0 à 19
        poser(i, 5, ALPHABET[i]);        // la lettre n° i, colonne i, ligne 5
    }

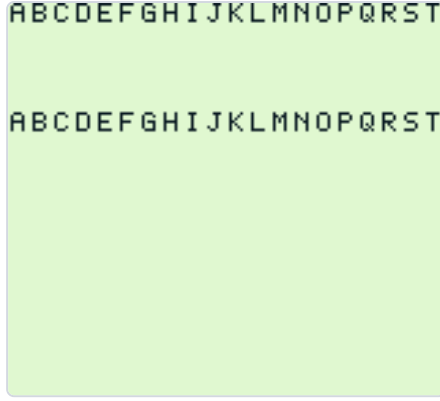
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 0 et sur la ligne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 0 : A à T

✓ la ligne 5 : A à T

Ensuite : **0.6. La même ligne, avec une boucle while** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++